

Correctness of the bit-blasted round-to-nearest-even circuit

A pen-and-paper proof, structured for mechanisation in Lean 4

Target file: Fp/Theorems/UnpackedFloat/Round.lean
theorem UnpackedFloat.toExtRat_round_Rel_smtLibRound_of_RNE

Abstract. We give a pen-and-paper proof that the bit-blasted round-to-nearest-ties-to-even circuit `blastSmtLibRound ... .RNE` computes a packed float that is `Rel`-equivalent to the SMT-LIB rounding specification `round .RNE`. Both sides are, structurally, the **same** four-way selection between two candidates — the greatest representable float $\leq r$ (**lower**) and the least representable float $\geq r$ (**upper**) — governed by the predicates `lowerHalf`, `tieBreak` and `isEven`. The proof therefore splits into two parts. **Part I** (Section 1–Section 4) proves the two candidates correct: `blastLower Rel lower` and, dually, `blastUpper Rel upper`. Its mathematical core (Section 3.3) is that bit-clearing implements truncation toward zero and that truncation is the greatest representable lower bound. We isolate the abstract specification interface (`IsLawfulLower`, `lower`, `Rel`, the float ordering) in Section 2 and treat it as a black box. **Part II** (Section 5) assembles RNE from those candidates: a single **selection engine** (Section 5.1) shows that the guard and sticky bits decide `lowerHalf` and `tieBreak` — stated purely as which of `lower/upper` is nearer, never via the round-toward-zero circuit — after which the four-way match and the boundary cases (zero, early overflow, early underflow) close the main theorem (Section 5.2).

1 Setup and notation

1.1 Formats and values

Fix two pairs of natural numbers. The **target format** (e_p, s_p) describes the packed floats we are rounding **into**; the **working format** (e_u, s_u) describes the unpacked float we are rounding **from**. We write

$$\text{PF} := \text{PackedFloat } e_p s_p, \quad \text{UF} := \text{UnpackedFloat } e_u s_u. \quad (1)$$

An unpacked float $x \in \text{UF}$ carries a sign bit $x.\text{sign}$, a signed exponent bit-vector $x.\text{ex} \in \text{BitVec } e_u$, and a significand bit-vector $x.\text{sig} \in \text{BitVec } s_u$. Its rational value (`UnpackedFloat.toRat'`, taken at precision p) is

$$x.\text{toRat}'(p) = (-1)^{x.\text{sign}} \cdot x.\text{sig}.\text{toNat} \cdot 2^{E(x,p)}, \quad E(x,p) = x.\text{ex}.\text{toInt} - p, \quad (2)$$

where $E(x,p)$ is `UnpackedFloat.toExpInt`. Throughout we take the default precision $p = s_u - 1$ and abbreviate $x.\text{toRat}' := x.\text{toRat}'(s_u - 1)$ and $E(x) := E(x, s_u - 1)$. We say x is **nonnegative** when $x.\text{sign} = \text{false}$.

A packed float $f \in \text{PF}$ has a rational/extended value $f.\text{toRat}$ and an extended-rational embedding $v(f) := f.\text{toExtRat} \in \overline{\mathbb{Q}}$, where $\overline{\mathbb{Q}} = \mathbb{Q} \cup \{\pm\infty, \text{NaN}\}$ is the type `ExtRat`. On non-special floats $v(f) = \text{Number } (f.\text{toRat})$.

1.2 The relevant constants

With $\text{bias}(e) = 2^{e-1} - 1$, recall (`Fp/Constants/Basic.lean`):

$$\begin{aligned} e_{\min}(e) &= -(\text{bias}(e) - 1) && \text{(minimum normal exponent)} \\ e_{\max}(e) &= \text{bias}(e) && \text{(maximum normal exponent)} \\ e_{\min}^{\text{sub}}(e, s) &= e_{\min}(e) - s && \text{(minimum subnormal exponent)}. \end{aligned} \quad (3)$$

The last identity is `minSubnormalExp_eq` unfolded: $e_{\min}^{\text{sub}} = (e_{\min} - 1) - (s - 1) = e_{\min} - s$. We also use the **exponent width** `exponentWidth( $e_p, s_p$ )`, the number of bits needed to hold every exponent of the target format including normalised subnormals; the standing hypothesis `exponentWidth( $e_p, s_p$ )  $\leq e_u$` guarantees the working format can name every target exponent.

1.3 The specification: **lower**

For $r \in \overline{\mathbb{Q}}$, the SMT-LIB greatest lower bound `smtLibLower.lower` is defined by Hilbert choice,

$$\text{lower } (r) := \epsilon x. \text{IsLawfulLower } (r, x), \quad (4)$$

where `(SmtLibSemantics.IsLawfulLower)`

$$\text{IsLawfulLower } (r, x) \iff v(x) \leq r \wedge (\forall x'. v(x') \leq r \implies x' \leq x). \quad (5)$$

That is, x embeds below r and is the $\leq$ -greatest such packed float, where $\leq$ is the float ordering (`PackedFloat.le`). The dual notion `IsLawfulUpper` and `upper` are defined symmetrically with the inequalities reversed.

1.4 The implementation: **blastLower**

The circuit `(Fp/UnpackedRound.lean)` is layered. The innermost piece,

$$\text{blastRoundTowardZero } (x) : \text{UnpackedFloat } e_u(s_p + 1), \quad (6)$$

clears the guard and sticky bits of x at the target precision and keeps the top $s_p + 1$ significand bits; we analyse it in Section 3.3.1. On top of it,

$$\text{blastLowerNonneg } (x) := \begin{cases} +0 & \text{if } \text{under}(x) \\ \text{maxNormal}_{e_u, s_p+1} & \text{else if } \text{over}(x) \\ \text{blastRoundTowardZero } (x) & \text{otherwise,} \end{cases} \quad (7)$$

where `under( $x$ ) := blastIsUnderflowNonneg( $x$ )` and `over( $x$ ) := blastIsEarlyOverflowNonneg( $x$ )` are the early under/overflow tests, returned as an `EUnpackedFloat` (an unpacked float extended with ∞/NaN states). Finally, splitting on the sign,

$$\text{blastLower } (x) := \begin{cases} -((-x).\text{blastUpperNonneg}) & \text{if } x.\text{sign} = \text{true} \\ \text{blastLowerNonneg } (x) & \text{otherwise,} \end{cases} \quad (8)$$

where `blastUpperNonneg` is the analogous least-upper-bound circuit.

1.5 The goal

The target relation `EUnpackedFloat.Rel` between $y : \text{EUnpackedFloat } e_u(s_p + 1)$ and $f : \text{PF}$ is

$$\begin{aligned} y \text{ Rel } f &\iff (y.\text{state} = \text{NaN} \wedge f.\text{isNaN}) \\ &\vee (y.\text{state} = \infty \wedge f.\text{isInfinite} \wedge y.\text{sign} = f.\text{sign}) \\ &\vee (y.\text{state} = \text{Number} \wedge y.\text{num}.\text{toRat}'(s_p) = f.\text{toRat} \wedge y.\text{num}.\text{sign} = f.\text{sign}). \end{aligned} \quad (9)$$

The theorem to prove `(UnpackedFloat.blastLower_Rel_smtLibLower)` is:

Theorem (Main). Assume $2 < e_p$, $0 < s_p$, $s_p + 2 \leq s_u$, `exponentWidth( $e_p, s_p$ )  $\leq e_u$` , and that x is normalised ($x.\text{normalize} = x$). Then

$$\text{blastLower } (x) \text{ Rel } (\text{lower } (\text{Number } (x.\text{toRat}')) : \text{PF}). \quad (10)$$

The standing hypotheses are used as follows. $0 < s_p$ and $2 < e_p$ make the target format non-degenerate (so `maxNormal` is not `NaN` and the bias is positive). `exponentWidth( $e_p, s_p$ )  $\leq e_u$` lets every target exponent be named in the working exponent width. $s_p + 2 \leq s_u$ leaves room for a guard bit

and a sticky region below the retained $s_p + 1$ bits (this is what makes the guard/sticky indexing in Section 3.3.1 well-typed and in-range). Normalisation $x.\text{normalize} = x$ pins down a **canonical** working representative, so that “keep the top $s_p + 1$ bits” is value-faithful — without it, several unpacked floats share a value and the bit-level truncation does not commute with Equation 2.

2 The abstract specification interface

We never unfold the Hilbert choice in `lower`. Instead we use only the following four facts about the specification and three facts about `Rel` and the ordering. In the mechanisation these are exactly the cited Lean lemmas; for the paper they are the axioms of our argument.

PROVED (S1) Lawfulness (`lsLawfulLower_smtLibLower`). For $0 < e_p$, $0 < s_p$ and any r , `lsLawfulLower (r, lower (r))`.

PROVED (S2) Uniqueness (`eq_of_IsLawfulLower_of_IsLawfulLower`). If $x, y : \text{PF}$ are both non-`NaN` and both satisfy `lsLawfulLower (r, ·)`, then $x = y$.

PROVED (S3) Non-`NaN` (`not_isNaN_lower_of_ne_NaN`). If $r \neq \text{NaN}$ then `lower (r)` is not `NaN`.

PROVED (S4) Negation duality (`smtLibLower_eq_neg_smtLibUpper_neg`). For $r \neq \text{NaN}$,

$$\text{lower } (r) = -(\text{upper } (-r)), \quad (11)$$

and dually `upper (r) = -(lower (-r))` (`smtLibUpper_eq_neg_smtLibLower_neg`).

PROVED (R1) Number introduction (`UnpackedFloat.Rel_of_toRat_eq_toRat_and_sign` composed with `EUnpackedFloat.Rel_of_Rel_of_state_eq_Number`). If $y.\text{state} = \text{Number}$, $y.\text{num}.\text{toRat}'(s_p) = f.\text{toRat}$ and $y.\text{num}.\text{sign} = f.\text{sign}$, then $y \text{ Rel } f$.

PROVED (R2) Negation is functorial (`EUnpackedFloat.neg_Rel_neg`). If $y \text{ Rel } f$ then $(-y) \text{ Rel } (-f)$.

PROVED (O1) Order reflection (`PackedFloat.le_of_toExtRat'_le_toExtRat'`). For non-`NaN` $f, g : \text{PF}$, if $v(f) \leq v(g)$ then $f \leq g$ in the float order. (Conversely `toExtRat'_le_toExtRat'_of_le` gives monotonicity; we only need reflection.)

The whole proof of the Main Theorem is now a finite case analysis that, in each leaf, **exhibits a witness** packed float f , proves `lsLawfulLower` of it, and matches values and signs. The next lemma is the engine that turns a witness into the conclusion.

PROVED Lemma 1 (Witness principle, `EUnpackedFloat.Rel_smtLibLower_of_witness`). Let $0 < e_p$, $0 < s_p$, $r \in \mathbb{Q}$, and let $y : \text{EUnpackedFloat } e_u(s_p + 1)$ have $y.\text{state} = \text{Number}$. Suppose there is $f : \text{PF}$ with

1. f is not `NaN`
2. `lsLawfulLower (Number (r), f)`;
3. $y.\text{num}.\text{toRat}'(s_p) = f.\text{toRat}$;
4. $y.\text{num}.\text{sign} = f.\text{sign}$.

Then $y \text{ Rel } (\text{lower } (\text{Number } (r))) : \text{PF}$.

Sketch. By (S1), `lower (Number (r))` is itself a lawful lower for `Number (r)`, and by (S3) it is non-`NaN` (as `Number (r) \neq NaN`). By hypotheses (1)–(2), f is a non-`NaN` lawful lower for the same r . Uniqueness (S2) forces $f = \text{lower } (\text{Number } (r))$. Now (3)–(4) and (R1) give $y \text{ Rel } f$, i.e. $y \text{ Rel } \text{lower } (\text{Number } (r))$. ■

It therefore remains to produce, in each branch of Equation 7 and Equation 8, a witness f together with proofs of (1)–(4). Branches (under/over) have easy explicit witnesses; the normal branch is the mathematical core.

3 The nonnegative case

Throughout this section assume $x.\text{sign} = \text{false}$, so $x.\text{toRat}' \geq 0$, and write $r := x.\text{toRat}' \geq 0$. We prove

$$\text{blastLowerNonneg } (x) \text{ Rel lower (Number } (r)) \quad (12)$$

(UnpackedFloat.blastLowerNonneg_Rel_smtLibLower) by the three-way split of Equation 7. Recall the exponent characterisations of the guards (already mechanised):

$$\begin{aligned} \text{under}(x) &= [x.\text{ex.toInt} < e_{\min}^{\text{sub}}(e_p, s_p)] \quad (\text{blastUnderflowNonneg_eq_decide}) \\ \text{over}(x) &= [e_{\max}(e_p) < x.\text{ex.toInt}] \quad (\text{blastIsEarlyOverflowNonneg_eq_decide}) \end{aligned} \quad (13)$$

where $[\cdot]$ is the Boolean indicator. Both require $1 < e_p$, $0 < s_p$, $\text{exponentWidth}(e_p, s_p) \leq e_u$.

3.1 Branch U: underflow

Witness: $f := \text{getZero}(e_p, s_p, \text{false}) = +0$.

Sketch. Assume $\text{under}(x)$, i.e. $x.\text{ex.toInt} < e_{\min}^{\text{sub}}$. By Equation 7 the circuit returns $+0$ (state Number). We apply Lemma 1 with witness $+0$. Conditions (1) $+0$ is not NaN, (3) $(+0).\text{toRat}'(s_p) = 0 = (+0).\text{toRat}$, and (4) signs are both false are immediate. The content is condition (2):

SORRY Lemma U (isLawfulLower_Number_getZero_of_underflowNonneg). If $x.\text{sign} = \text{false}$ and $\text{under}(x)$, then $\text{isLawfulLower}(\text{Number } (r), +0)$.

For Lemma U we must show $v(+0) = 0 \leq r$ (true since $r \geq 0$) and that every f' with $v(f') \leq r$ has $f' \leq +0$. Here the underflow bound Equation 13 is the crux: $x.\text{ex.toInt} < e_{\min}^{\text{sub}} = e_{\min} - s_p$ together with Equation 2 forces $0 \leq r < 2^{e_{\min}^{\text{sub}}} =$ (the smallest positive representable magnitude). Hence the only representable f' with $v(f') \leq r$ are $\leq +0$ in the float order: a positive representable float has value $\geq 2^{e_{\min}^{\text{sub}}} > r$, contradicting $v(f') \leq r$, so f' is a zero or negative, all $\leq +0$. ■

3.2 Branch O: overflow

Witness: $f := \text{maxNormalNumber}(e_p, s_p, \text{false})$, the largest finite float.

Sketch. Assume $\text{over}(x)$, i.e. $e_{\max} < x.\text{ex.toInt}$. By Equation 7 the circuit returns the unpacked $\text{maxNormal}_{e_u, s_p+1}$. Apply Lemma 1 with witness maxNormalNumber . Condition (1): its exponent is $\text{allOnes} - 1 \neq \text{allOnes}$, so it is not NaN (PackedFloat.not_isNaN_maxNormalNumber, needs $0 < e_p$). Condition (3): the unpacked and packed maxima agree under value,

$$(\text{maxNormal}_{e_u, s_p+1}).\text{toRat}'(s_p) = (\text{maxNormalNumber}).\text{toRat} \quad (14)$$

(UnpackedFloat.toRat'_maxNormal_eq_toRat_maxNormalNumber, needs $0 < s_p$, $s_p + 1 \leq s_u$, $2 < e_p$, $\text{exponentWidth} \leq e_u$). Condition (4): both signs false. The content is condition (2):

SORRY Lemma O (isLawfulLower_Number_maxNormalNumber_of_overflowNonneg). If $x.\text{sign} = \text{false}$ and $\text{over}(x)$, then $\text{isLawfulLower}(\text{Number } (r), \text{maxNormalNumber})$.

For Lemma O: the overflow bound $e_{\max} < x.\text{ex.toInt}$ gives, via Equation 2, $r > 2^{e_{\max}+1} \geq \text{value of maxNormalNumber}$, so $v(\text{maxNormalNumber}) \leq r$. For maximality, any representable f' with $v(f') \leq r$ is finite (since $+\infty > r$ is excluded only if...) — more precisely every **finite** float is $\leq \text{maxNormalNumber}$ in value, and $+\infty$ has $v(+\infty) = +\infty > r$ because $r \in \mathbb{Q}$; hence $f' \leq \text{maxNormalNumber}$ by (O1). ■

Remark on bounds. Overflow uses the **early** test $e_{\max} < x.\text{ex.toInt}$ rather than a value comparison against maxNormalNumber . This is sound for **lower** because once the exponent strictly exceeds $e_{\max}$, the value exceeds

every finite float regardless of significand, so `maxNormalNumber` is the greatest representable lower bound. (For `upper` the same exponent regime instead yields $+\infty$; see Section 4.)

3.3 Branch N: the normal range

This is the heart of the proof. Assume $x.\text{sign} = \text{false}$, $x.\text{normalize} = x$, $\neg \text{under}(x)$ and $\neg \text{over}(x)$; by Equation 13 this means

$$e_{\min}^{\text{sub}}(e_p, s_p) \leq x.\text{ex.toInt} \leq e_{\max}(e_p). \quad (15)$$

By Equation 7 the circuit returns `blastRoundTowardZero (x)` (state `Number`). The goal is

SORRY Lemma N (`exists_packedFloat_isLawfulLower_of_blastRoundTowardZero`). Under the hypotheses above there is $f : \text{PF}$ with f non-NaN, `isLawfulLower (Number (r), f)`, `(blastRoundTowardZero (x)).toRat' = f.toRat`, and `(blastRoundTowardZero (x)).sign = f.sign`.

Given Lemma N, the normal branch follows from Lemma 1 immediately (`UnpackedFloat.blastLowerNonneg_Rel_smtLibLower_normal`). We now prove Lemma N by reducing it to three sublemmas about truncation toward zero.

3.3.1 What `blastRoundTowardZero` computes

The circuit is `blastRoundTowardZero (x) = (blastClearSig (x))` with its top $s_p + 1$ significand bits extracted, where `blastClearSig (blastClearSignificand)` zeroes all significand bits at and below the **guard index**

$$g(x) := (s_u - 1) - (s_p + 1) + (e_{\min}(e_p) - x.\text{ex.toInt.toNat}) \quad (16)$$

(`UnpackedFloat.toNat_guardBitIndex_eq`). The natural-number coercion $(e_{\min} - x.\text{ex.toInt.toNat})$ is the **subnormal shift** $\delta(x)$: the number of extra low-order significand positions that must be discarded when x sits below the minimum normal exponent. For g to be a sensible index and δ to behave, the indexing lemma needs three bounds, all discharged here from Equation 15 and the standing hypotheses:

$$-2^{e_u-1} \leq e_{\min} - x.\text{ex.toInt} < 2^{e_u-1}, \quad \delta(x) = (e_{\min} - x.\text{ex.toInt.toNat}) \leq s_p. \quad (17)$$

The right-hand bound $\delta(x) \leq s_p$ is **exactly** $\neg \text{under}(x)$: by Equation 3, $e_{\min} - x.\text{ex.toInt} \leq s_p \iff x.\text{ex.toInt} \geq e_{\min} - s_p = e_{\min}^{\text{sub}}$. The two-sided bound holds because $e_{\min} - x.\text{ex.toInt}$ fits in the signed working exponent width (using `exponentWidth` $\leq e_u$ and Equation 15). These are the hypotheses `hdiffLower`, `hdiffUpper`, `hdiffNatLe` that thread through every guard/sticky lemma in the file.

SORRY Claim (within one ulp). Let $t := (\text{blastRoundTowardZero } (x)).\text{toRat}'(s_p)$ and write $\beta := 2^{x.\text{ex.toInt} - s_p + \delta(x)}$ for the target unit in the last place. Then t is a representable value at most one ulp below r :

$$0 \leq r - t < \beta. \quad (18)$$

We do **not** need the exact quotient $t = \beta \lfloor r/\beta \rfloor$; the lower-bound $t \leq r$ and the strict gap $r - t < \beta$ are all that the maximality argument (N3) consumes. In the normal regime $x.\text{ex.toInt} \geq e_{\min}$ the shift vanishes ($\delta(x) = 0$) and $\beta = 2^{x.\text{ex.toInt} - s_p}$ is the ordinary normal ulp; in the subnormal regime $\delta(x) = e_{\min} - x.\text{ex.toInt} > 0$ raises the exponent back to the fixed subnormal spacing $\beta = 2^{e_{\min} - s_p}$.

Sketch. By Equation 2, $r = x.\text{sig.toNat} \cdot 2^{E(x)}$. Clearing the bits at and below index $g(x)$ replaces $x.\text{sig.toNat}$ by a multiple of $2^{g(x)+1}$ that is $\leq x.\text{sig.toNat}$ (`blastClearSignificand_sig_toNat_le`), and discards a tail strictly smaller than $2^{g(x)+1}$. Extracting the top $s_p + 1$ bits and re-reading at precision s_p rescales by a fixed power of two, giving the lower bound $t \leq r$ (`blastClearSignificand_toRat_le_of_nonneg`, with nonnegativity of t from $x.\text{sign} = \text{false}$) and the strict gap $r - t < \beta$. ■

The use of $x.\text{normalize} = x$ enters here: normalisation guarantees the implicit leading one sits in the top bit, so that “keep the top $s_p + 1$ bits” retains the hidden bit plus s_p significand bits and the discarded tail is genuinely below one ulp — making the strict gap $r - t < \beta$ of Equation 18 hold.

3.3.2 Truncation is the greatest representable lower bound

We now manufacture the witness f . Because Equation 15 holds, t from Equation 18 has an exponent within the target range, so it is **representable**:

SORRY Sublemma N1 (representability). There is $f : \text{PF}$, non-NaN, with $f.\text{toRat} = t$ and $f.\text{sign} = (\text{blastRoundTowardZero } (x)).\text{sign} = \text{false}$.

Proof idea. t has the form $m \cdot 2^{x.\text{ex.toInt} - (s_p - 1) - \delta}$ with $0 \leq m < 2^{s_p + 1}$ an integer significand (the top $s_p + 1$ bits), and exponent in $[e_{\min}^{\text{sub}}, e_{\max}]$ by Equation 15 and $\delta \leq s_p$. Packing m and the exponent produces f ; that pack is value- and sign-preserving is **Packing-style bookkeeping**. (In the file this f is obtained existentially; one may instead take $f := (\text{blastRoundTowardZero } (x)).\text{pack}$.) ■

PROVED Sublemma N2 (lower bound). $v(f) = \text{Number } (t) \leq \text{Number } (r)$.

Proof. Immediate from $t \leq r$ in Equation 18 and v on finite floats being **Number** $(\cdot)$. ■

SORRY Sublemma N3 (maximality). For every $f' : \text{PF}$ with $v(f') \leq \text{Number } (r)$, we have $f' \leq f$.

Proof. Let $v(f') \leq \text{Number } (r)$. Then f' is finite (as **Number** $(r) \neq +\infty$ and $\neq \text{NaN}$), so $v(f') = \text{Number } (f'.\text{toRat})$ with $f'.\text{toRat} \leq r$. Every representable value is an integer multiple of its own unit-in-last-place, and at the exponent of f the target grid has spacing β ; since $f'.\text{toRat} \leq r$ and $f'.\text{toRat}$ is a grid point, while $f = t$ is the grid point with $t \leq r < t + \beta$ by the one-ulp gap Equation 18, no grid point lies in $(t, r]$, so $f'.\text{toRat} \leq t = f.\text{toRat}$. Thus $v(f') \leq v(f)$, and (O1) gives $f' \leq f$. ■

SORRY Proof of Lemma N. Take f from N1. It is non-NaN. N2 and N3 are exactly the two conjuncts of Equation 5, so **IsLawfulLower** $(\text{Number } (r), f)$. Finally $(\text{blastRoundTowardZero } (x)).\text{toRat}' = t = f.\text{toRat}$ (N1) and $(\text{blastRoundTowardZero } (x)).\text{sign} = \text{false} = f.\text{sign}$ (N1). ■

Where the bounds are essential. N3 needs “the next grid point above t exceeds r ,” which is $r - t < \beta$ from Equation 18. That strict one-ulp gap depends on the guard index $g(x)$ pointing at the correct bit, which in turn requires Equation 17 — i.e. the full strength of $\neg \text{under}(x)$, $\neg \text{over}(x)$, $s_p + 2 \leq s_u$, and $\text{exponentWidth}(e_p, s_p) \leq e_u$. Drop any of these and either $g(x)$ leaves the significand window or δ saturates, breaking Equation 18.

4 The sign reduction and the Main Theorem

It remains to remove the standing assumption $x.\text{sign} = \text{false}$ used in Section 3, and to assemble the pieces. The negative case is handled by duality, never re-doing the grid argument.

SORRY Lemma U★ (blastUpperNonneg_Rel_smtLibUpper). For $x.\text{sign} = \text{false}$ (and the standing hypotheses), **blastUpperNonneg** $(x) \text{ Rel upper } (\text{Number } (x.\text{toRat}'))$.

Lemma U★ is the mirror image of Section 3: the same three branches with the inequalities reversed, the witnesses **minSubnormal** (underflow), $+\infty$ (overflow), and **blastSuccessorAwayFromZero** (x) (normal). We take it as established by the symmetric development (it is the dual sorry in the file).

SORRY Proof of the Main Theorem (**UnpackedFloat.blastLower_Rel_smtLibLower**). Split on $x.\text{sign}$. (*Structure complete; depends on the SORRY lemmas above.*)

- **$x.\text{sign} = \text{false}$.** Then $\text{blastLower}(x) = \text{blastLowerNonneg}(x)$ by Equation 8, and the conclusion is `blastLowerNonneg_Rel_smtLibLower`: combine Branches U, O, N above through Equation 7. Each branch supplies a witness and invokes Lemma 1; the guard characterisations Equation 13 select the branch matching the specification's own case analysis.

- **$x.\text{sign} = \text{true}$.** Then $\text{blastLower}(x) = -((-x).\text{blastUpperNonneg})$ by Equation 8. Since $x.\text{toRat}' \neq \text{NaN}$, the duality (S4) gives

$$\text{lower}(\text{Number}(x.\text{toRat}')) = -(\text{upper}(-\text{Number}(x.\text{toRat}')) = -(\text{upper}(\text{Number}((-x).\text{toRat}'))),$$

using $-\text{Number}(x.\text{toRat}') = \text{Number}((-x).\text{toRat}')(\text{toRat_neg})$. Now $-x$ has sign `false`, so Lemma U★ yields $(-x).\text{blastUpperNonneg} \text{ Rel } \text{upper}(\text{Number}((-x).\text{toRat}'))$, and (R2) negates both sides:

$$-((-x).\text{blastUpperNonneg}) \text{ Rel } -(\text{upper}(\text{Number}((-x).\text{toRat}')) = \text{lower}(\text{Number}(x.\text{toRat}'))$$

This is precisely $\text{blastLower}(x) \text{ Rel } \text{lower}(\text{Number}(x.\text{toRat}'))$. ■

This completes **Part I**. The same development run on the dual circuit yields the second candidate, `UnpackedFloat.blastUpper_Rel_smtLibUpper`:

$$\text{blastUpper}(x) \text{ Rel } \text{upper}(\text{Number}(x.\text{toRat}')), \quad (21)$$

where `blastUpper` is `blastUpper`. From here on we treat both $\text{blastLower}(x) \text{ Rel } \text{lower}(r)$ and $\text{blastUpper}(x) \text{ Rel } \text{upper}(r)$ as available.

5 From lower and upper to round-to-nearest-even

We now assemble RNE. The point of departure is that the implementation and the specification are the **same** four-way selection. The SMT-LIB rounder (`SmtLibSemantics.RoundMethod.roundRNE`) is

$$\text{roundRNE}(\text{sign}, r) := \begin{cases} \text{upper}(r) & \text{if } \neg \text{lowerHalf}(r) \wedge \neg \text{tieBreak}(r) \\ \text{upper}(r) & \text{if } \text{tieBreak}(r) \wedge \text{isEven}(\text{upper}(r)) \\ \text{lower}(r) & \text{if } \text{tieBreak}(r) \wedge \text{isEven}(\text{lower}(r)) \\ \text{lower}(r) & \text{if } \text{lowerHalf}(r) \end{cases} \quad (22)$$

(plus the zero case, handled by the wrapper below), and the circuit (`UnpackedFloat.blastSmtLibRoundRNE`) is the identical table with $\text{lower} \mapsto \text{blastLower}$, $\text{upper} \mapsto \text{blastUpper}$, and the predicates replaced by their bit-blasted counterparts `blastIsLowerHalf`, `blastIsTieBreak`, `blastIsEvenLower`, `blastIsEvenUpper`. With Part I supplying the two candidates, RNE reduces to matching the **predicates**, then to a mechanical case match, then to the boundary wrapper.

5.1 The selection engine

We phrase the selection **entirely in terms of the two candidates** $\text{lower}(r)$ and $\text{upper}(r)$ from Part I — never in terms of the internal round-toward-zero circuit. This keeps Part II at the specification level and lets the negative branch be recovered by the same negation duality (S4) used in Section 4: for $x.\text{sign} = \text{true}$ we work with $-x$ (nonnegative) and read off $\text{lower}(r) = -(\text{upper}(-r))$, so the entire engine is developed once, for the nonnegative branch, around `lower`.

Assume the nonnegative normal range of Section 3.3. There $\text{lower}(r)$ and $\text{upper}(r)$ are the two adjacent representable points bracketing r , with

$$(\text{lower}(r)).\text{toRat} \leq r \leq (\text{upper}(r)).\text{toRat} . \quad (23)$$

“Round to nearest” asks which endpoint is closer, and a tie is the exact midpoint. The guard and sticky bits are precisely the bits that decide this:

SORRY **Lemma E (selection engine).** In the nonnegative normal range,

$$\begin{aligned} \text{blastIsLowerHalf}(x) &= [r - (\text{lower}(r)).\text{toRat} < (\text{upper}(r)).\text{toRat} - r] \quad (\text{blastIsLowerHalf_eq_decide_dist_lt_of_nonneg}) \\ \text{blastIsTieBreak}(x) &= [r - (\text{lower}(r)).\text{toRat} = (\text{upper}(r)).\text{toRat} - r] \quad (\text{blastIsTieBreak_eq_decide_dist_eq_of_nonneg}). \end{aligned} \quad (24)$$

That is, **lowerHalf** is “ r strictly nearer **lower**”, and **tieBreak** is “ r equidistant from **lower** and **upper**”.

Why this is the keystone. Lemma E is the only genuinely new mathematical content of Part II. Its proof is where the half-ulp arithmetic lives — the guard bit sits at value $((\text{upper}(r)).\text{toRat} - (\text{lower}(r)).\text{toRat})/2$, the midpoint of Equation 23 — but the **statement** exposes nothing but **lower** and **upper**. Everything below is bookkeeping on top of Lemma E and Part I.

From Lemma E the two SMT-LIB selector bridges follow, because **lowerHalf** and **tieBreak** are **defined** (`SmtLibSemantics.smtLibRoundMethod`) by comparing **lower** at precisions s_p and $s_p + 1$ — i.e. by exactly the nearer-endpoint test of Lemma E, identified through the Part I specifications:

SORRY (P1) (`blastIsLowerHalf_iff_smtLibLowerHalf`). $\text{blastIsLowerHalf}(x) = \text{true} \iff \text{lowerHalf}(\text{Number}(r)).$

SORRY (P2) (`blastTieBreak_iff_smtLibTieBreak`). $\text{blastIsTieBreak}(x) = \text{true} \iff \text{tieBreak}(\text{Number}(r)).$

PROVED (P3) (`blastIsEvenLower_eq`, `blastIsEvenUpper_eq`). The even predicates already match: $\text{blastIsEvenLower}(x)$ and $\text{blastIsEvenUpper}(x)$ equal $\text{isEven}(\text{lower}(\text{Number}(r)))$ and $\text{isEven}(\text{upper}(\text{Number}(r)))$ respectively.

The sign reduction for **lowerHalf/tieBreak** is the same negation-duality device as Section 4 (the negative case mirrors guard/sticky on $-x$), so Lemma E is stated for the nonnegative branch and lifted.

5.2 The boundary wrapper and the Main Theorem

The top-level circuit `UnpackedFloat.blastSmtLibRound ... .RNE` guards the selection of Equation 22 with three early exits, matched against the specification’s own special cases:

SORRY (B0) **Zero.** If x is zero the circuit returns ± 0 ; the spec agrees via `roundRNE_zero`.

SORRY (B+) **Early overflow** (`roundRNE_eq_infinity_of_maxNormalExp_lt`). If $e_{\max} < x.\text{ex.toInt}$ the circuit returns $\pm\infty$ and so does `roundRNE` (RNE sends everything above the largest finite float to infinity).

SORRY (B−) **Early underflow** (`roundRNE_eq_zero_of_lt_minNormalExp`). Below the minimum subnormal exponent the circuit returns ± 0 and so does `roundRNE`.

Off the boundary, `truncateFittingExponent` re-seats the exponent without changing the **Rel**-class (`blastLower_truncateFittingExponent_Rel_eq_blastLower_Rel` and its **blastUpper** twin), and we are in the regime where Lemma E and (P1)–(P3) apply. The four-way match of Equation 22 against the circuit is then mechanical.

SORRY **Main Theorem (RNE)** (`UnpackedFloat.toExtRat_round_Rel_smtLibRound_of_RNE`).

For a normalised x and the standing hypotheses,

$$(\text{blastLower} / \text{blastUpper} \text{ selection on } x) \text{ Rel round } (\text{RNE}, x.\text{sign}, \text{Number}(x.\text{toRat})). \quad (25)$$

Proof. Case on the wrapper. (B0)/(B+)/(B−) discharge the boundary exits. Otherwise rewrite the circuit’s predicates by (P1)–(P3), rewrite the spec by Equation 22, and match the four branches: each returns **blastLower** (x) or **blastUpper** (x), which are **Rel**-correct by Part I. ■

6 Dependency map for mechanisation

The following table lists every lemma the argument consumes, its status in `Round.lean`, and the section that uses it. “Interface” lemmas are assumed; the remaining ones (Lemmas U, O, N, the sublemmas, and the Part II obligations) are the genuine proof obligations, currently `sorry` in the file.

Used in	Lemma (Lean name)	Status
Lemma 1	<code>EUnpackedFloat.Rel_smtLibLower_of_witness</code>	PROVED
S1	<code>lsLawfulLower_smtLibLower</code>	PROVED
S2	<code>eq_of_IsLawfulLower_of_IsLawfulLower</code>	PROVED
S3	<code>not_isNaN_lower_of_ne_NaN</code>	PROVED
S4	<code>smtLibLower_eq_neg_smtLibUpper_neg</code>	PROVED
R1	<code>Rel_of_toRat_eq_toRat_and_sign, Rel_of_Rel_of_state_eq_Number</code>	PROVED
R2	<code>EUnpackedFloat.neg_Rel_neg</code>	PROVED
O1	<code>PackedFloat.le_of_toExtRat'_le_toExtRat'</code>	PROVED
Equation 13	<code>blastUnderflowNonneg_eq_decide, blastIsEarlyOverflowNonneg_eq_decide</code>	PROVED
Branch U (Lemma U)	<code>isLawfulLower_Number_getZero_of_underflowNonneg</code>	SORRY
Branch O (Lemma O)	<code>isLawfulLower_Number_maxNormalNumber_of_overflowNonneg</code>	SORRY
Branch O (support)	<code>not_isNaN_maxNormalNumber, toRat'_maxNormal_eq_toRat_maxNormalNumber</code>	PROVED
Branch N (Lemma N)	<code>exists_packedFloat_isLawfulLower_of_blastRoundTowardZero</code>	SORRY
Equation 18 ($\leq$ part)	<code>blastClearSignificand_toRat_le_of_nonneg, blastClearSignificand_sig_toNat_le</code>	PROVED
Equation 18 (strict gap)	the $r - t < \beta$ bound — strengthen the $\leq$ -lemmas to a strict one-ulp gap	SORRY
Equation 17	<code>toNat_guardBitIndex_eq, toNat_lsbIndex_eq</code>	PROVED
Lemma U★	<code>blastUpperNonneg_Rel_smtLibUpper</code>	SORRY
Part I main	<code>blastLower_Rel_smtLibLower, blastUpper_Rel_smtLibUpper</code>	SORRY
Part II — round-to-nearest-even (Section 5)		
Lemma E (Section 5.1)	<code>blastIsLowerHalf_eq_decide_dist_lt_of_nonneg, blastIsTieBreak_eq_decide_dist_eq_of_nonneg</code>	SORRY
P1	<code>blastIsLowerHalf_iff_smtLibLowerHalf</code>	SORRY
P2	<code>blastTieBreak_iff_smtLibTieBreak</code>	SORRY
P3	<code>blastIsEvenLower_eq, blastIsEvenUpper_eq</code>	PROVED
B0	<code>roundRNE_zero</code>	PROVED
B+	<code>roundRNE_eq_infinity_of_maxNormalExp_lt</code>	SORRY
B−	<code>roundRNE_eq_zero_of_lt_minNormalExp</code>	SORRY
Re-seat exponent	<code>blastLower_truncateFittingExponent_Rel_eq_blastLower_Rel, truncateFittingExponent_Rel_eq_blastUpper_Rel</code>	SORRY
Main (RNE)	<code>toExtRat_round_Rel_smtLibRound_of_RNE</code>	SORRY

PROVED interface fact or already in `Round.lean`. **SORRY** remaining proof obligation (currently `sorry`).

6.1 Recommended mechanisation order

The steps below are ordered by dependency: each consumes only proved facts and the outputs of earlier steps. Steps 1–5 are **Part I** (the two candidates); Steps 6–8 are **Part II** (RNE). Equation numbers refer to the boxed, numbered displays above.

6.1.1 Step 1 — the one-ulp gap (Equation 18)

The normal branch needs only that `blastRoundTowardZero (x)` lands within one ulp below r . The file already proves the lower bound $t \leq r$ (`blastClearSignificand_toRat_le_of_nonneg`, `blastClearSignificand_sig_toNat_le`); add the **strict gap**

$$0 \leq r - (\text{blastRoundTowardZero } (x)).\text{toRat}'(s_p) < \beta, \quad \beta = 2^{x.\text{ex.toInt} - s_p + \delta(x)}. \quad (26)$$

We deliberately do **not** prove the exact quotient $t = \beta \lfloor r/\beta \rfloor$: the maximality argument (N3) consumes only $t \leq r$ and $r - t < \beta$, so we stay at the level of **lower/upper** rather than naming a round-toward-zero value. Prove the gap at the bit level: `blastClearSignificand` discards the bits at and below the guard index $g = g(x)$, a tail strictly below 2^{g+1} ; rescaling by the top- $(s_{p\{+\}}1)$ -bit extraction turns this into $r - t < \beta$. The index arithmetic needs the three bounds of Equation 17, already available via `toNat_guardBitIndex_eq`. **This step requires** $x.\text{normalize} = x$ (the hidden bit in the top position is what makes the discarded tail genuinely below one ulp).

6.1.2 Step 2 — Sublemmas N1, N3 and Lemma N (Equation 18, Equation 15, Equation 17)

With Step 1 in hand, N2 is already trivial. Prove N1 (representability) by packing the integer significand m and the in-range exponent guaranteed by Equation 15 together with $\delta(x) \leq s_p$ from Equation 17; the cleanest witness is $f := (\text{blastRoundTowardZero } (x)).\text{pack}$, so that the existential in `exists_packedFloat_isLawfulLower_of_blastRoundTowardZero` becomes a concrete term. Prove N3 (maximality) from the strict bound $x.\text{toRat}' - t < \beta$ of Equation 18 plus the grid-spacing fact and order reflection (O1). Assembling N1–N3 through Equation 5 discharges Lemma N.

6.1.3 Step 3 — the boundary branches, Lemmas U and O (Equation 13)

Both are short once Equation 13 rewrites the Boolean guards as exponent comparisons. For Lemma U, $x.\text{ex.toInt} < e_{\min}^{\text{sub}}$ forces $0 \leq x.\text{toRat}' < 2^{e_{\min}^{\text{sub}}}$, below the smallest positive representable, so `+0` is the greatest lower bound. For Lemma O, $e_{\max} < x.\text{ex.toInt}$ forces $x.\text{toRat}'$ above every finite float, so `maxNormal` is the greatest lower bound; the value/sign side conditions are already proved (`toRat'_maxNormal_eq_toRat_maxNormalNumber`, `not_isNaN_maxNormalNumber`).

6.1.4 Step 4 — mirror to the upper circuit, Lemma U★

Repeat Steps 1–3 with all inequalities reversed to obtain `blastUpperNonneg_Rel_smtLibUpper`: the dual primitive is `blastSuccessorAwayFromZero` (round **away** from zero), and the boundary witnesses become `minSubnormal` (underflow) and `+∞` (overflow). The one-ulp gap of Step 1, dualized to a gap **above** r ($0 \leq t' - r < \beta$ for the rounded-up value t'), gives the dual of Equation 18. No new ideas are needed.

6.1.5 Step 5 — assembly of the candidates (Equation 7, Equation 8)

The top-level case analysis is already written in `blastLower_Rel_smtLibLower`. The nonnegative case combines Steps 2–3 through Equation 7; the negative case rewrites via the duality (S4) and Equation 8, applies Step 4, and negates with (R2). Once Steps 1–4 are filled, this theorem — and dually `blastUpper_Rel_smtLibUpper` — close with no remaining `sorry`. This finishes Part I.

6.1.6 Step 6 — the selection engine, Lemma E (Section 5.1, Equation 23)

The genuinely new Part II content. Prove `blastIsLowerHalf_eq_decide_dist_lt_of_nonneg` and `blastIsTieBreak_eq_decide_dist_eq_of_nonneg`: that the guard bit decides “ r nearer **lower**” and guard-with-clear-sticky decides “ r at the midpoint”, both stated as comparisons of the distances $r - (\text{lower } (r)).\text{toRat}$ and $(\text{upper } (r)).\text{toRat} - r$. The arithmetic core is that the guard bit sits at the midpoint of Equation 23; this is where the half-ulp reasoning lives, but it is discharged **once**, here, and never leaks the round-toward-zero circuit into the statement. The negative branch is the (S4) duality, as in Section 4.

6.1.7 Step 7 — the selector bridges P1, P2 (Equation 22)

With Lemma E and the Part I specifications, prove `blastIsLowerHalf_iff_smtLibLowerHalf` and `blastTieBreak_iff_smtLibTieBreak` by unfolding the SMT-LIB definitions of `lowerHalf/tieBreak` (the two-precision `lower` comparison) to the nearer-endpoint test of Lemma E. The even predicates P3 (`blastIsEvenLower_eq`, `blastIsEvenUpper_eq`) are already proved.

6.1.8 Step 8 — the boundary wrapper and the RNE theorem (Section 5.2)

Discharge the three early exits — B0 zero (`roundRNE_zero`, proved), B+ early overflow (`roundRNE_eq_infinity_of_maxNormalExp_lt`), B− early underflow (`roundRNE_eq_zero_of_lt_minNormalExp`) — and the exponent re-seating (`blastLower_truncateFittingExponent_Rel_eq_blastLower_Rel` and its `blastUpper` twin). Then in `toExtRat_round_Rel_smtLibRound_of_RNE`, rewrite the circuit’s predicates by Steps 6–7 (P1–P3), rewrite the spec by Equation 22, and match the four branches — each returning `blastLower` or `blastUpper`, correct by Part I. Once Steps 1–7 are filled, this closes the development with no remaining sorry.